

Pre-defense transcript

Md. Sakif Khan

Rehearsal script for ****pre-defense-0421052099.pdf**** — 31 pages: a title, twenty-nine body slides, a closing slide.

The six backup slides are a **separate file**, **pre-defense-0421052099-backup.pdf**. You do not present them. Open it alongside the main deck and jump to a slide when a question calls for one; the table near the end of this file maps each to its question.

Budget: 23 min 56 s of speaking against a 25-minute limit. That leaves just over a minute, and every row below is set to exactly what its own words take at 93 wpm — so the minute is the whole of the slack, and no individual slide holds any.

The table read 24:26 until its rows were checked against the words above them, and fourteen of the rows demanded a faster rate than this script's own — the echo-attractor slide worst, at 125 wpm against 93. Every round's additions had been costed against the total and never against the row they landed in, so the table stayed internally consistent while drifting away from the speech it describes. It is re-derived from the words on every edit now.

The seven crowded slides became seventeen this round, and the background the new audience needs — what a hallucination is, where it comes from, what a knowledge graph is — is new speech, not redistributed speech. It cost 2 min 39 s on the first draft, which ran 25:40. All of it came back out of the seventeen sections that had just been written: no section that was already settled and already timed gave up a word, and the total moved from 22:41 to 23:58 rather than past the limit.

A minute is a margin, not a cushion. Everything that could move to the answers has already moved there. If you need more than the minute, it has to come out of what is said: about 60 words buys 40 seconds, and the recovery notes below name the slides that can give it up.

Times below are *cumulative at the end of that slide*. If you are more than 40 seconds past a marker, use the recovery notes at the bottom.

#	Slide	Slide time	Cumulative
1	Title	0:19	0:19
2	The problem	0:32	0:51
3	What a hallucination is	0:38	1:29
4	Where it comes from	0:31	2:00
5	Which of the three can we act on?	0:32	2:32
6	What a knowledge graph is	0:37	3:09
7	The complication: mediator nodes	0:34	3:43
8	Where existing approaches stop	0:41	4:24
9	What everyone else does	0:56	5:20
10	Research questions	0:53	6:13
11	AGR: an explicit state machine	1:08	7:21
12	Constrained tools	0:37	7:58
13	The Structural Verification Layer	0:32	8:30
14	What <i>structural</i> means — and what it does not	0:58	9:28
15	One claim, three routes	1:00	10:28
16	One question, end to end	1:08	11:36
17	The environment and the question sets	0:59	12:35
18	Making the comparison fair	0:32	13:07
19	What is <i>not</i> held equal	0:43	13:50
20	Main results	1:10	15:00
21	RQ1: accuracy against hop count	1:02	16:02
22	The caveat I want to raise myself	1:05	17:07
23	RQ2: does anything ungrounded get asserted?	1:03	18:10
24	RQ2: what verification does <i>not</i> do	0:45	18:55
25	RQ2: so what does it do?	0:41	19:36
26	RQ3: what each component earns	0:22	19:58
27	RQ3: one effect, and its sign is backwards	0:48	20:46
28	Every failure, read and labelled	0:24	21:10
29	The echo attractor	1:12	22:22
30	Contributions, and what I would not claim	1:29	23:51
31	Thank you	0:05	23:56

The four **bold** slides are the ones the committee will actually interrogate. If you are running long, take time from 8, 12, and 15 — never from 20, 21, 22, 26 or 27. That is the same list the recovery notes protect, and 22 is on it for a different reason: skipping it hands the clipping issue to them.

1 — Title

(0:19)

Good morning. I'm Sakif Khan. This is my pre-defense on Agentic Graph Reasoning — knowledge graph navigation with verification before the answer is emitted. My supervisor is Dr. Sadia Sharmin.

Don't read the title aloud. It's on the screen.

2 — The problem

(0:32)

Language models answer factual questions fluently — and just as fluently when they do not hold the fact. Fluency and factuality come out of the same mechanism, so nothing in the wording separates them. A system that is confidently wrong is worse than one that says it does not know.

3 — What a hallucination is

(0:38)

First, the word. A hallucination is fluent, confident, and backed by no source the model can point to.

Two kinds, separated by what it takes to catch them. Factually wrong: you need the true answer already, which is what we are trying to produce. Unsupported: you need only what was retrieved, and we have that. I claim the second.

Name the two block titles as you say them — the slide carries the rest. The last sentence is a scope limit, not modesty. Say it at normal pace.

4 — Where it comes from

(0:31)

Why does it happen? Not a bug that escaped testing — it comes out of how these models are built, so the response has to be architectural.

Three origins. The training data. The model itself, where knowledge sits spread across the weights with no index. And the prompt.

Name the three, don't read them. The slide carries the wording.

5 — Which of the three can we act on?

(0:32)

Only the third; the other two need retraining. So this is a systems problem: control what goes in, and check what comes back out against it.

Multi-hop compounds it. A wrong first step is never revisited, so the chain proceeds from a false premise. That is where this thesis lives.

6 — What a knowledge graph is

(0:37)

Two things about the graph, because they shape everything after.

Facts are stored as triples — head, relation, tail. Ada Lovelace, place of birth, London. A question needing two of those chained is multi-hop. The point of a graph is that every fact has an address, so a claim can be checked by looking for an edge.

7 — The complication: mediator nodes

(0:34)

Freebase stores any fact with more than two participants as a node rather than an edge. “Rainn Wilson played Dwight in The Office” is one node joining three things. Nearly two thirds of this graph’s nodes are that kind, and they carry no name.

So nothing here is called “plays”. Hold onto that.

The last line is a setup. The worked example pays it off, and so does the verification layer’s honest limit.

8 — Where existing approaches stop

(0:41)

Four families, and each stops somewhere specific — the table walks left to right. The first three stop before reasoning begins, or at a radius fixed in advance. Agentic navigation does interleave retrieval and reasoning, which is the right move.

But all four share one gap: whatever the final generation call produces is what gets emitted. Nothing checks what the answer *asserts*.

Walk the table left to right with the pointer. Don’t read the cells verbatim.

9 — What everyone else does

(0:56)

Five systems, and the column that matters is the last one.

The four prior systems differ in how they explore — a fixed radius, generated paths, beam search, adaptive planning. Some decompose the question, one backtracks. None checks what its own answer asserts against what it retrieved, before answering. That empty column is where this thesis sits.

One caution. Their published accuracies are higher than anything I am about to show, and not comparable: different backbones, different subsets, and full Freebase rather than the environment I built.

Scope every count to the four prior systems. Backtracking is PoG's alone among them; "two can backtrack" counted AGR in one breath and "none checks" counted it out in the next, and the table on screen shows both columns.

If a committee member knows these papers, this is where they will ask about the numbers. The environment slide answers it.

10 — Research questions

(0:53)

Three questions.

RQ1: does agentic navigation actually improve multi-hop accuracy — and does the advantage grow with the number of hops?

RQ2: given a system that already navigates the graph, what does a claim-level check against the traversed triples contribute?

RQ3: which components earn their cost, in accuracy and in tokens?

One framing note. RQ2 asks what verification *contributes*, not whether it reduces hallucination. That's deliberate. The answer has several parts, and a yes-or-no question would have hidden most of them.

11 — AGR: an explicit state machine

(1:08)

AGR is an explicit state machine — not a prompt loop. Six nodes.

The planner decomposes the question into ordered sub-objectives. The explorer scores candidate edges and expands the frontier. The evaluator decides whether the sub-objective has been met. The backtracker undoes a bad expansion and bans the edge that caused it. The verifier is the contribution. The answerer emits only what survived.

Three cycles — the three arrows returning to the explorer — and all three are bounded by explicit budgets rather than by model behaviour. Every cycle passes through a router checking a monotone counter, so termination does not depend on the model cooperating.

Do not say “exactly two cycles.” The diagram has three arrows returning to the Explorer and the audience is looking at it while you speak. The thesis caption says two and names two, but the figure source calls the third one a cycle in its own comment. Naming them after the edge labels — continue, backtrack, retry — means counting the arrows confirms the sentence.

If asked about budgets, go to backup page 2.

12 — Constrained tools

(0:37)

One agent, six nodes — and only three touch the graph. The planner links entities, the explorer asks for relations and neighbours, the verifier checks adjacency. Four operations, never Cypher. Every call is deterministic and logged with its arguments and result, so the traversal is a record — and that record is what the verification layer checks against.

Say “one agent” out loud. The previous slide draws six named boxes, and “the agent” on its own invites the question of how many there are: they are six nodes of one machine over one shared state, not six agents. The Node column answers the follow-up before it is asked.

13 — The Structural Verification Layer

(0:32)

This is the contribution.

Before anything is emitted, the draft is split into atomic claims, each checked against the triples the agent actually traversed. What cannot be grounded is re-explored or dropped; what is grounded carries its triples back with the answer. So the system hedges rather than asserts.

14 — What *structural* means — and what it does not (0:58)

Now the bound.

Structural means the check is against the graph the agent walked, not the model's opinion of its own output. It does not mean the check reads the relation: the first two routes test adjacency, either direction, so a mother claim survives on a child edge — the mediator problem from slide 7, as a limitation.

“Its evidence” is narrower too: only the walked-graph check attaches triples, and the log keeps a count. Slide 24 has both. And a claim can be true and still be the wrong answer.

The last sentence is a promise. Slide 29 pays it off.

15 — One claim, three routes (1:00)

This is the inside of the verifier — the node between the evaluator deciding to answer and the answer going out.

It drafts an answer and breaks it into atomic claims, one model call. Each claim then takes one of three routes to being called supported, ordered by cost: only the third spends a model call. Plus-evidence sits on the first route only.

The box at the bottom is how the node exits: no unsupported claims means grounded and the answer goes out; otherwise retry if the budget allows, or give up.

Say the first sentence before anything else. This is the one slide in the deck that is a zoom rather than a new subject, and without that framing it reads as a second, unrelated flowchart next to slide 11's. The bottom box is the same three edges leaving the Verifier there: grounded and give\{\}_up go to the Answerer, retry goes back to the Explorer.

16 — One question, end to end

(1:08)

One real question, all the way through — a run out of the committed records.

“Who plays Dwight in The Office.” The planner splits it in two and resolves The Office to a node. The explorer scores the available relations and takes the regular-cast one; the evaluator resolves Dwight Schrute. Second hop: the character relation, and the evaluator resolves Rainn Wilson and answers. The verifier splits the draft into two claims, both supported.

Six model calls, depth two, eighteen supporting triples.

And look at that second relation: the path runs *through* a mediator, which is why it is named for the appearance rather than for “plays”.

If you are behind, cut everything after “decides to answer.”

17 — The environment and the question sets

(0:59)

The environment is a Freebase-derived graph: 2.6 million entities, 8.3 million triples, seven thousand distinct relations, importing in 36 seconds.

Questions: 400 each from WebQSP and ComplexWebQuestions. The important column is reachability — for every question I verified the gold answer is actually reachable here. 97 percent on WebQSP, 99.2 on CWQ.

That is the *ceiling* on every accuracy figure I’m about to show. It is also the answer to the caution two slides back: an environment this reachable is not full Freebase, so these numbers are not comparable with published ones.

18 — Making the comparison fair

(0:32)

Five systems, on the slide: a parametric control, three retrieval baselines, and AGR.

All five run on one frozen backbone, at temperature zero, under the same 25-call budget, on the same questions, against the same graph. That is what lets me attribute differences to architecture rather than to model capacity.

19 — What is *not* held equal

(0:43)

One thing is not equal, and I would rather say it than have it found. Think-on-Graph prunes from a narrower candidate set than AGR. That cuts against it, not for it — a thinner set is cheaper — so it cannot explain the clipping, but it does make its score a lower bound.

And the parametric control is there because these benchmarks may be partly memorised.

Say the width line, don't skip it. The slide claimed a retrieval-budget control it does not have; the Think-on-Graph baseline section names the widths as the one place a reader should look first for a confound, and the conclusion ranks it limitation 5. The numbers are on the slide and in the answer below — you do not have to recite 40/20/300/200 here.

20 — Main results ★

(1:10)

Here is the comparison.

AGR reaches 0.755 Hits@1 and 0.642 F1 on WebQSP, and 0.522 and 0.469 on ComplexWebQuestions — ahead of every baseline on both.

Two things beyond the top line. First, vector RAG on ComplexWebQuestions: 0.203, *below* the no-retrieval control at 0.307. One verbalised triple cannot contain a chain, so single-shot retrieval is worse there than not retrieving at all. GraphRAG sits beside it at 0.205, but its one-hop radius confounds the paradigm, so the claim rests on vector RAG.

Second, the cost columns: 4,511 tokens and 6.2 calls against Think-on-Graph's 3,615 and 12.8. More tokens, half the calls — and calls are what the budget meters.

Slow down here. This is the slide they read while you talk.

Do not pool the two retrieval baselines. The table puts 0.203 and 0.205 side by side and the pooled version of this point is the one that gets walked back: results.tex calls GraphRAG the weaker evidence of the two and says the claim rests on vector RAG. The paper retracted the pooled claim in its own words. If they press on GraphRAG, the answer below has the strata.

21 — RQ1: accuracy against hop count ★

(1:02)

The direct answer to RQ1, and it is a shape rather than a difference of means.

ComplexWebQuestions, on the right. AGR goes 0.46, 0.55, 0.57 as questions get harder — one hop, two, three or more. It is the only system on that dataset that ends above where it started. Three of the other four decay monotonically; Think-on-Graph falls and partially recovers, still 0.08 below its own one-hop score.

One caution about the left panel: on WebQSP the three-or-more stratum is n equals 4. I draw no conclusion from four questions, and neither does the thesis.

Volunteering the $n=4$ weakness pre-empts the obvious attack.

22 — The caveat I want to raise myself

(1:05)

I want to raise this before you do.

Split the questions by whether Think-on-Graph finished inside the shared 25-call cap. On the ones it *finishes*, it is ahead of AGR — 0.852 against 0.788 on WebQSP, 0.629 against 0.607 on CWQ.

The entire aggregate margin comes from the questions it cannot finish, where it drops to 0.197 and 0.188. It gets clipped on 29 percent of WebQSP and 44 percent of CWQ.

So the honest claim is not that AGR reasons better per step. It is that AGR completes its reasoning inside a fixed budget and Think-on-Graph frequently does not.

Deliver this as a strength. It is the most defensible slide in the deck.

23 — RQ2: does anything ungrounded get asserted?

(1:03)

RQ2. Does anything ungrounded get asserted?

Pooling both datasets: AGR asserts 1,709 entities and zero are absent from the graph. The parametric control asserts 1,001 and 22.1 percent don't exist.

That looks like a headline result for the verification layer. It isn't, and this is the finding I think matters most: Think-on-Graph also reaches 0.0 percent, and it has no verification layer at all.

So zero ungrounded assertion is a property of *navigating a graph* — you can only name what you visited. It is not a property of my layer, and I don't claim it as one.

24 — RQ2: what verification does *not* do

(0:45)

Which leaves the real question. The negative half first: removing the layer changes accuracy by an amount I cannot detect — p equals 1.0 on both datasets. I report that as a negative result.

And the record does not survive the run: the logger writes the count of supporting triples and drops the list. You can confirm the system tracked its evidence; you cannot open the record and inspect it.

25 — RQ2: so what does it do?

(0:41)

What it does do is withhold what it cannot ground. Removing the layer drops the CWQ hedge rate from 23.2 to 20.2 percent — direction only, six questions. A hedge is a calibrated non-assertion, not a refusal.

It attaches supporting triples, from one route of three, and pairs the answer with its evidence inside the run. So the case rests on auditability, not accuracy.

26 — RQ3: what each component earns★

(0:22)

RQ3: which components earn their cost.

Four ablations, paired McNemar against the full system. Three of the four change nothing I can detect — backtracking, the verifier, and the learned half of the scorer.

27 — RQ3: one effect, and its sign is backwards★

(0:48)

The one that does is the planner, and the sign is backwards. Removing it *improves* WebQSP by 0.083 F1 at p equals 0.006, and cuts tokens by 31 percent. On ComplexWebQuestions it trends the other way.

The explanation is hop count: WebQSP is mostly one-hop, and decomposing a one-hop question sends the frontier away from the answer. So decomposition should be gated on question structure — a conclusion the measurement forced, not one it confirmed.

Do not compress this one. It is the only place a component's cost is priced.

28 — Every failure, read and labelled

(0:24)

I read all 259 remaining failures and labelled them by hand. Top categories, pooled across both datasets.

The largest is relation selection — the agent taking the wrong edge, not inventing one. That shape is the finding.

The table is pooled, and the slide says so. Wrong and hedge are never pooled in the thesis — sec:taxonomy: “a pooled percentage would describe neither” — and pooling also hides the shape flip: `composite_claim` is 1 on WebQSP against 46 on CWQ. The caption carries both facts and points at backup page 4, so the split census is something you offer rather than something you are corrected with.

The full 12-category histogram is backup page 4 if anyone wants it.

29 — The echo attractor

(1:12)

The one to name is sixth, with 13 cases: the **echo attractor** — a real, grounded entity one hop from the correct answer. Ask for a director and get the film; ask for a capital and get the country.

Why it matters, and it is the promise I made on slide 14: any grounding check *passes* it. It is real, it was traversed, the triple exists — true, and wrong.

Different systems fall into it together, so no evaluation treating them as independent can see it. Rescore whenever a majority agrees — a natural thing to want — and this becomes apparent correctness. That is the contribution: the mechanism, not the count.

Do not say “it appears across systems, so it is a property of the task rather than of AGR.” That is defensive where the thesis is substantive, and nothing on the slide blames AGR for it. The claim is about evaluation: sec:echo calls the attractor “invisible to any evaluation treating systems as independent”, and section 1.6 says the contribution is the mechanism “and what it means for consensus-based evaluation, not the frequency”. Backup page 7 is the same finding seen from the other side.

30 — Contributions, and what I would not claim

(1:29)

To summarise. Six contributions, the six the thesis claims in section 1.6. The one I'd underline is stratum-dependent decomposition: the literature treats it as straightforwardly beneficial, and it is not.

The sixth is worded *pre-registered* in the thesis; nothing was filed with a registry, so I say *pre-specified*.

The limitations, which I'd rather state than be asked. The first is the most serious: the verifier persists only what it *rejects*, so wrongful acceptance has no rate at all — and the same decision is why the output contract cannot be audited from the record.

Then: no detectable accuracy gain, and at n around 200 that is “not detected”, not “none”. Zero ungrounded assertion comes from navigation, not my layer. One environment, one backbone, one annotator. And Think-on-Graph leads where it finishes, from a narrower candidate set.

31 — Thank you

(0:05)

Thank you. I'm happy to take questions.

Backup slides

A separate document. Open it before you start, minimised or on a second screen. **Everything in this script refers to backup slides by the page number the viewer shows**, which is what this table lists. The file opens on a title page, so the first backup slide is page 2. An ordinal would resolve one short of every one of them: counted that way, the fourth backup slide is hedging rather than the census.

Page	Contents	Use when asked
2	Budget configuration and enforcement sites	“How do you guarantee termination?”
3	Which budgets actually bind	“Is the 25-call cap fair to Think-on-Graph?”
4	Full 12-category failure histogram	“What were the other failure modes?”
5	Hedging rates, all five systems	“Doesn't it just refuse more often?”
6	Accuracy against cost, both metrics	“Is it cheaper, or just better?”
7	The benchmark was wrong 57 times	“How good is the gold?”

Page 3 is the important one. It shows AGR never reaches the call cap — 0.0 percent on both datasets — which is precisely what makes the comparison against a clipped Think-on-Graph legitimate rather than an artefact of a cap chosen to suit AGR.

Anticipated questions

“How do you know models actually do this? Where is your evidence?” From this work’s own no-retrieval baseline, which is the parametric control in the five-system comparison. On WebQSP it asserted 661 entities and 179 of them do not exist anywhere in the knowledge graph — 27.1 percent, every one stated without a hedge. Slide 23 gives the same measurement over both datasets pooled: 1,001 asserted, 221 ungrounded, 22.1 percent. The WebQSP slice runs higher because CWQ questions are longer and the model hedges more on them; the pooled row is the one I quote, and it is the one in the thesis.

“Isn’t the 25-call cap arbitrary, and doesn’t it favour AGR?” It’s the cap Think-on-Graph’s own paper operates under. And AGR never reaches it — zero percent on both datasets (backup page 3). If I raised the cap, AGR’s numbers would not move; Think-on-Graph’s would. I say that in the thesis rather than leaving it for someone to find.

“Do the categories look the same on both datasets?” (*Slide 28 is pooled.*) No, and that is the more interesting answer. The census is reported split in the thesis and never pooled, because wrong and hedge describe different failure semantics and the proportions differ sharply by dataset. The clearest case is `composite_claim`: 1 on WebQSP against 46 on CWQ. WebQSP questions mostly are not compound, so the category barely exists there; on ComplexWebQuestions it is the largest single failure mode. A pooled percentage would describe neither dataset. The full split is backup page 4.

“Doesn’t GraphRAG show the same thing?” (*Slide 20 puts 0.203 and 0.205 side by side. Do not pool them.*) No, and the thesis says so rather than letting the two numbers be read together. GraphRAG retrieves a one-logical-hop neighbourhood, so its fall on CWQ confounds static retrieval with a radius I chose in advance. The evidence that it is the radius is in the strata: GraphRAG scores 0.44 on WebQSP’s two-hop questions, which are largely mediator paths a one-hop expansion does reach, against 0.16 on the CWQ two-hop stratum, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap. There is a second bound on it: it takes at most 100 edges per topic entity with no ordering imposed, and on 72.5 percent of questions at least one topic entity exceeds that degree, so on those it answers from an arbitrary sample. Vector RAG carries the paradigm claim, because one verbalised triple cannot contain a chain at any radius.

“Did both systems see the same candidate sets?” (*The sharper form of the cap question. Slide 19 raises it deliberately — answer it, don’t deflect.*) No, and it is the one thing I do not hold constant. Think-on-Graph keeps 40 relations per entity and 20 neighbours per relation — the pruning widths of the algorithm as published — where AGR keeps 300 and 200. Measured over the committed tool logs, the 40-relation cut binds on 31.6 percent of the 1,651 entities it expanded and the 20-neighbour cut on 32.8 percent of its neighbour calls; AGR’s relation cap binds once in 3,097 expansions and its neighbour cap on 3.3 percent of calls. So the cut is real and it is asymmetric. What it cannot do

is rescue the budget argument: a narrower candidate set makes each step *cheaper*, so it cannot explain why Think-on-Graph runs out of calls. What it does mean is that the residual gap on the questions it *finishes* is measured against a system searching a thinner pool, so that figure is a lower bound on what it could resolve at equal width — not an estimate of it. Re-running it at 300 and 200 is the first item in my future work, and it is limitation 5 in the conclusion.

“Your verifier doesn’t verify the relationship — any edge between the two entities passes.” Correct, and I would rather name it than defend it: it is the layer’s principal acceptance risk, and section 6.8 lists it first among the mechanisms of wrongful acceptance. Both structural routes ignore the relation and the direction, so a claim that X is Y’s mother is certified by an edge recording that X is Y’s child. It is a deliberate consequence of one design choice. The claim’s relation is free text and Freebase predicates are not — “X played for Y” is realised through a roster mediator carrying no predicate named for playing — so matching on it would reject true claims wholesale. The division of labour is that the structural routes buy recall for “some supporting edge exists” and the third route, the entailment check, buys the semantic precision. What I will not do is state the exposure at the smaller of the two scales available. The population at risk is not the rare `verify_connection` route but every claim those two routes accept between them, and the log does not separate them: on test, of 2,008 accepted claims, somewhere between 39 and all 2,008 were certified without any test of the asserted relation. That is an interval two orders of magnitude wide and I report it as one rather than choose a point inside it. Narrowing it needs the logging change — persist accepted claims with the triples that matched them — which is the same future-work item as the previous answer, and it is what would let the relation be checked where the mediator schema makes that possible.

“If verification doesn’t improve accuracy, why keep it?” Because accuracy was never the only claim. It converts silent error into an explicit hedge, and it pairs the answer with the triples that ground it at the point of emission. I report the null on accuracy rather than hiding it — and I report the bounds on the auditability too, which is the next question.

“Show me one supporting triple, then.” (*Expect this. Slide 23 invites it.*) I can’t, from the committed record, and that is a limitation rather than an evasion. `RunLogger` writes `n_supporting_triples` — an integer — and discards the list, so no committed artifact in this work contains a single supporting triple; every statistic I quote about them comes from that counter. What I can show is the traversal record each answer was produced against. Two things follow and I state both: the pairing is real inside the run and unavailable afterwards, and one polarity of the verifier’s error is therefore unmeasured. Persisting accepted claims with their matching triples is one logging change, and it is the first item in my future work. I did not make it late because it would separate the code from results already frozen against it.

“Why isn’t the five-system comparison one of your contributions?” Because the thesis does not count it as one, and slide 30 is the thesis’s list — section 1.6, one subsection per item. The comparison and the hop-count shape are results the contributions rest on; they get slides 20 and 21, which is where the weight belongs. An earlier version of that slide promoted both to contributions and dropped the ablation, the decomposition finding and the protocol to make room — still saying “six”. The thesis had already been audited for exactly that mismatch, where its conclusion counted four against section 1.6’s six.

“How many questions is that hedge difference, and is it significant?” Six, on CWQ: 23.2 percent of 198 against 20.2. The sets nest — there is no question the ablated run hedged on that the full system asserted on — so those are exactly the six the ablated run answered and the layer declined to. None of the six came back correct: all six were assertions that would have been wrong, which is the mechanism, seen at the only place the design isolates it.

On WebQSP it is one question, and there the ablated run got it right. So across the 398 paired questions correctness moved twice, once each way. I would rather say that than be shown it. No, it is not significance-tested; the ablation’s McNemar test is on correctness, not on the hedge column, and I claim the direction and nothing more. *Do not* offer the no-retrieval contrast here. Its 12.2 percent is a hedge rate, not an error rate — backup slide 5 has it in a column headed “WebQSP hedge %” — and AGR hedges *less* than it does, 8.2 against 12.2, so that comparison argues the opposite of what it looks like. No-retrieval’s actual error rate is 170 wrong out of the 351 questions it asserts on.

“Does every accepted claim get evidence attached?” No — one route of three does. Traversed adjacency attaches every traversed triple joining the pair; `verify_connection` and the entailment fallback accept with nothing attached. On the 80-question development set, 13 answers carry no supporting triples: ten are hedges that asserted nothing, one had every claim rejected, and two asserted a single claim certified by the route that records no evidence. Median across all 80 is 3, mean 4.1, maximum 16 — read from the counter, since the triples are exactly what the log does not keep.

“Your planner result says your own design is wrong.” It says the planner is wrong *for one-hop questions*, and WebQSP is mostly one-hop. On ComplexWebQuestions the effect reverses in sign. The conclusion is that decomposition should be gated on question structure — which is a finding, and one I’d have missed without the ablation.

“n=4 in the three-hop WebQSP stratum is meaningless.” Agreed, and I don’t draw a conclusion from it. The hop-count claim rests on ComplexWebQuestions, where the strata are 137, 211, and 49.

“How do you know the gold answers are reachable?” Verified per question against the graph before scoring — 97.0 percent on WebQSP, 99.2 on CWQ. It’s the ceiling on every Hits@1 I report, and it’s stated as such.

“Where do the topic entities come from — doesn’t the system have to find them first?” They are given by the datasets. `use_gold_entities` is on for every run I report, so the question’s annotated mentions go to `search_entity`, which resolves each one to a graph node through the three-stage resolver. Mention *detection* is assumed; mention-to-node resolution is not — that part the system does. It is limitation 7 in the conclusion: the accuracies I report presume a linking step a deployed system would have to perform, and I do not measure that step. What it is not is a between-system confound. The three systems that touch the graph — AGR, Think-on-Graph and GraphRAG — all seed from the same annotated mentions, and neither the parametric control nor Vector-RAG ever sees them.

“Nine of your failures are one bug. Isn’t the census measuring your implementation?” In part, and I would rather say which part. Nine of the 38 `decomposition_error` cases carry the subtype `extraction_bug`, and they share one mechanism: the evaluator resolves every gold value, the drafted sentence names them verbatim, and `answer_entities` then collapses to the sentence’s grammatical subject. WebQTest-1215 drafts all six of

Stephen Covey's professions and scores ['Stephen Covey']. The reasoning was complete and the verifier certified it; what failed is the step that reads entities back out of the draft. That is limitation 8, and the direction is the part worth saying: it depresses my *own* reported accuracy, so on that question shape the headline numbers are a floor rather than an estimate. The fix is an instruction to the claim decomposer, not an architecture change, and it is the first of the three repairs the census earned. The category split is backup page 4.

“Is this reproducible?” Every number in the thesis is generated from frozen run records by a script; none is transcribed. Same for the three data figures in this deck — they are pulled directly from the thesis's own generated sources.

Recovery notes

If you hit **15:00 (the end of slide 20) more than 40 seconds late**, compress as follows.

- Slides 24 and 25 — take them as one: *“It doesn't improve accuracy. It converts silent error into an explicit hedge and attaches evidence. The case is auditability.”* Saves ~50 s.
- Slide 4 — name the three origins and stop; skip the model sentence. Saves ~15 s. Slide 8 has already been compressed this way and cannot give again.
- Slide 3 — the two kinds are on the slide; say the last sentence only. Saves ~20 s. This is the one addition of this round that the talk can lose without losing a claim.

Never compress 20, 21, 22, 26, or 27. Slide 22 in particular: skipping it means a committee member raises the clipping issue instead of you, and it lands very differently that way.

Delivery

- **Say the number, then what it means.** Not the reverse. “Twenty-seven percent of asserted entities don't exist — the model is confidently inventing.”
- **Four slides are negative results** (21, 22, 23, 24). Deliver them at normal pace, not apologetically. A student who reports a clean null is more credible than one who reports only wins.
- The word is **hedge**, not “refuse.” A hedge is a calibrated non-assertion.
- If you lose your place, the takeaway bar at the bottom of the data slides is your prompt — read it aloud and continue.